

x402, the Agent Capability Market, and **h402**.

Why agents still cannot buy anything, and what four competing payment standards are doing about it.

Every capability is a small project of its own.

Payment is not even the first wall. This is what one API costs you, before a single useful call happens.

find	find	find	find	find	find	find	find	find	find
docs	docs	docs	docs	docs	docs	docs	docs	docs	docs
sdk	sdk	sdk	sdk	sdk	sdk	sdk	sdk	sdk	sdk
shape	shape	shape	shape	shape	shape	shape	shape	shape	shape
auth	auth	auth	auth	auth	auth	auth	auth	auth	auth
errors	errors	errors	errors	errors	errors	errors	errors	errors	errors
test	test	test	test	test	test	test	test	test	test

TEN APIS, BEFORE THE FIRST CALL

70

And then it still cannot pay.

Signing up is a human act. Every step below assumes a person is sitting there.

WHAT THE API ASKS FOR

create an account
verify an email
store an API key
put a card on file
accept the terms
pass a captcha

WHAT THE AGENT CAN DO

none of it
none of it
none of it
none of it
none of it
none of it

so it stops and asks you,
and you are back in the loop.

HTTP had a status code for this **since 1997.**

402 Payment Required sat reserved and unused for almost thirty years. x402 fills it in. Coinbase published it, and it moved to the Linux Foundation in April 2026.

```
$ GET /routes/nansen/crypto/smart-money-flows
HTTP/1.1 402 Payment Required { "amount": "52500", "asset": "USDC", "network": "eip155:8453" }
$ GET /routes/nansen/crypto/smart-money-flows X-PAYMENT: <signed EIP-3009 authorization>
HTTP/1.1 200 OK { "data": [ ... ] }
```

Four steps. About two seconds. No human anywhere in it.

01 • ASK

Call the route

The server answers 402 with the exact price, before anything is charged.

PRICE KNOWN UP FRONT

02 • SIGN

Authorize offchain

An EIP-3009 signature. No transaction from the agent, no gas.

BUYER GAS \$0

03 • SETTLE

Facilitator settles

It verifies the signature and moves the USDC onchain.

FEE FREE ON BASE

04 • DELIVER

Result comes back

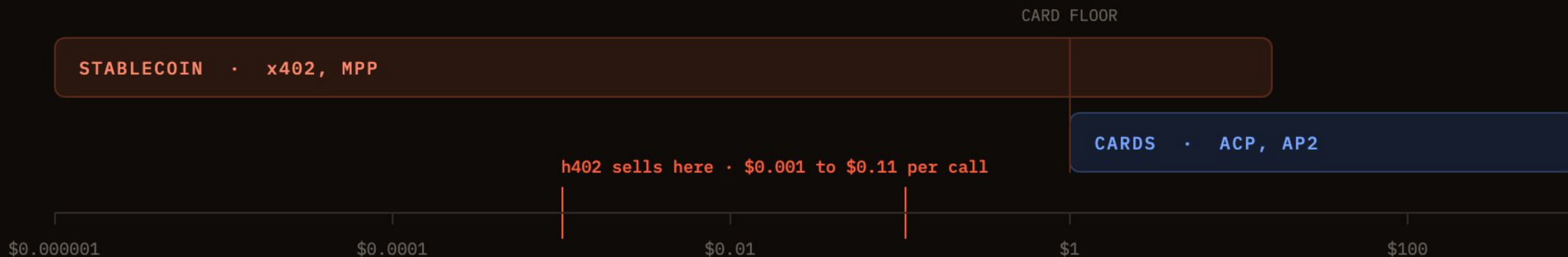
With a payment reference, so the call can be audited later.

RECEIPT ATTACHED

The agent holds its own wallet and its own per-call spend cap the whole time.

The market did not split along web2 and web3. It split **by transaction size.**

Card payments carry a fixed cost per transaction. On a \$20 purchase that is noise. On a \$0.002 API call it is a hundred times the price, and no negotiation fixes a floor.



WHY STABLECOIN GOES LOW

No fixed cost per authorization, and batching pushes it lower still.
Circle's Nanopayments settles down to \$0.000001.

WHY CARDS STAY HIGH

Above a few dollars you want chargebacks, dispute handling and merchants that already accept you. Stablecoin rails have none of that yet.

Four standards. **Two different jobs.**

MACHINES BUYING FROM MACHINES

APIs, compute, data

Per call, tiny amounts, no human in the loop.



x402

Coinbase, now Linux Foundation

T MPP

Stripe + Tempo, Mar 2026

AGENTS BUYING FROM SHOPS

Goods, checkouts, subscriptions

Dollars, a cardholder behind it, merchants that already exist.



ACP

with OpenAI, ChatGPT checkout



AP2

Google identity and payments

THE ONE NUMBER ANYONE PUBLISHES

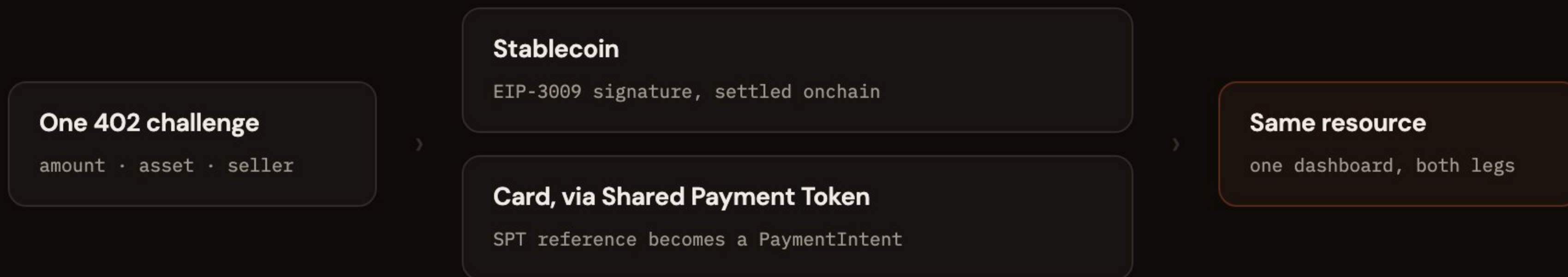
x402 has processed around **165 million calls** and roughly **\$50M** cumulative. The other three have not published volume at all.

AND THE ONE THAT MATTERS LATER

Only x402 is governed outside the company that wrote it. The other three are vendor or co-authored.

Everyone else picked a side. Stripe built both, then **bridged them.**

ACP with OpenAI for the card band. MPP with Tempo for the machine band. Then one 402 challenge that either rail can settle.



Both legs land in the same Stripe dashboard as PaymentIntents, so a merchant reconciles once. And the card leg leans on a Shared Payment Token, which is worth a closer look.

Two payment worlds built the **same primitive**.

A one time, scoped permission to move a fixed amount to a fixed counterparty. Stripe holds one. The agent signs the other.

S SHARED PAYMENT TOKEN

scope	one seller
cap	one amount
life	expires, revocable
held by	Stripe
rails	card, BNPL

EIP-3009 AUTHORIZATION

scope	one payTo address
cap	one amount
life	validAfter to validBefore
held by	the agent
rails	any USDC chain

Same idea, two custodial models. One asks you to trust a company. The other asks you to hold a key. That choice, not the protocol, is what actually splits this market.

Discovery is solved.

Selection is not.

Everyone shipped a directory. A directory tells an agent what exists. It does not tell it which one works.

A TYPICAL LISTING SHOWS	NAME	PRICE	ENDPOINTS	DOES IT WORK
Circle agent marketplace	yes	yes	yes	no signal
Aggregator listings on it	yes	flat rate	yes	no signal
Provider docs	yes	yes	yes	their own claim

WHAT IS NEVER SHOWN

Success rate. Median latency. Price against a competitor for the same task. Whether the endpoint answered at all this week.

SO THE AGENT

still has to choose, and has nothing to choose with. It picks the first listing, or the cheapest, and finds out the hard way.

Mount it once. The agent can buy **anything in the catalog.**

One call path per capability. No keys, no accounts, no subscriptions, no SDK per provider. Every call is a real USDC payment on Base, quoted before it is charged.

```
/routes/{provider}/{category}/{action} // the hundredth capability is the same shape as the first
```

CATALOG

~140

capabilities across 16 categories

PROVIDERS

71

contracts competing on shared
paths

VERIFIED

100%

of enabled routes, by real paid
calls

SETTLES ON

Base

USDC over x402, per call

A listing is a claim. A settled payment is **evidence**.

We buy every route with our own money before we list it, and keep buying it after. The score that decides which provider gets recommended comes from those calls.

01

Buy it first

Nothing is listed until we have paid for it and seen a real response.

02

Measure

Success rate, median latency, price. From live paid traffic, not docs.

03

Rank

Providers on the same task are scored against each other.

04

Pin one

The agent is handed a default, so it never has to shop.

EVERY LATER CALL FEEDS THE NEXT SCORE

A list can be copied in a week. A year of settled calls cannot.

The protocol war is not the agent's problem **to solve.**

The agent

one call, one price, one
result

x402 on Base

buyer payment, and most provider payouts

MPP on Tempo

when a provider speaks nothing else

One response

the agent never knew

Agents should not have to pick a payment standard, any more than a browser picks a TLS version.

h402 settles buyer payments over x402 on Base. MPP only candidates settle on Tempo. Same call either way.

Fresh agent session. One link. **Real money.**

Nothing installed. Watch two things: what it decides to buy, and the price it shows you before it pays.

Capabilities it was never told to combine, for **a few cents**.

NOBODY WIRED THAT

It needed one thing to get another, read the catalog, and bought them in the right order. No pipeline, no integration, no glue code written by us or by you.

TWO WORLDS, ONE SESSION

Regulated filing data and onchain liquidity data, from different providers, joined in one answer. Normally that is two subscriptions and a spreadsheet.

THE PRICE WAS ALWAYS VISIBLE

Every call quoted the exact charge before signing. Nothing was spent silently.

WHOSE ANSWER IS IT

The data belongs to the providers. The read on top of it is the agent's. Worth saying out loud, because that line matters.

Stop asking what your agent knows. Ask **what it can buy**.

If you build agents

The design problem moves from prompts to budgets, spend caps, and which provider to trust. That is a surface nobody has good defaults for yet.

If you have an API

Agents are a customer segment that cannot sign up for anything. A middleware and about a day is the whole lift to start selling to them.

If you are picking a protocol

None of the four has won, and they are not fighting over the same transactions. Build behind an abstraction and let the price band decide the rail.